AN EVIDENCE-ADAPTIVE, OPTICAL CHARACTER RECOGNITION (OCR)-RESILIENT HYBRID RETRIEVAL FRAMEWORK FOR DOCUMENT QUESTION ANSWERING

A Project Report

Submitted by

MANTHIRA MOORTHY P

VIJAY S K

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI

(An Autonomous Institution affiliated to Anna University Chennai)

CERTIFICATE

This is to certify that this report entitled **"An Evidence-Adaptive, OCR-Resilient Hybrid Retrieval Framework for Document Question Answering"** is the bonafide work done by **Manthira Moorthy P** and **Vijay S K** of the Department of Electronics and Communication Engineering, Mepco Schlenk Engineering College, Sivakasi, under the guidance of **Dr. V. Karthikeyan, Assistant Professor (Sl. Grade)**.

Note on this certificate page: institutional signature fields (Faculty In-Charge, Head of the Department, Internal Examiners, Principal) and the official submission date are intentionally left as placeholders below — they were not supplied to this workflow and must be filled in and signed by the actual institutional authorities before submission. The project guide's name is taken directly from the project's own source documentation (`PROJECT_SCOPE.md`).

..... | .....

Faculty In-Charge | Head of the Department

Submitted for the practical examination held on [DATE TO BE FILLED]

..... | .....

Internal Examiner I | Internal Examiner II

ACKNOWLEDGEMENT

The authors thank the Department of Electronics and Communication Engineering, Mepco Schlenk Engineering College, Sivakasi, for the facilities extended for this project. The authors express sincere gratitude to their project guide, **Dr. V. Karthikeyan, Assistant Professor (Sl. Grade)**, for guidance throughout the project. **[Principal's name and Head of Department's name to be inserted here — not available to this workflow].**

ABSTRACT

Retrieval-augmented question answering (QA) systems retrieve top-k passages and generate an answer unconditionally, regardless of whether the retrieved evidence actually supports a correct answer — including QuIM-RAG (Saha et al., IEEE Access, 2024), the question-indexed retrieval baseline this project extends. This is especially risky for scanned, Optical Character Recognition (OCR)-processed documents, where recognition noise, broken layouts, and ambiguous retrieval make silent wrong answers more likely. This project implements and evaluates an OCR-resilient hybrid-retrieval document QA system that adds Evidence-Adaptive Retrieval and Verification (EAR): a calibrated decision layer combining four signals already available at retrieval time — dense/lexical retrieval agreement, propagated OCR confidence, cross-passage numeric contradiction, and query complexity — into a logistic sufficiency score, used to decide whether to answer, expand retrieval, inspect structure, or abstain. On the OHR-Bench benchmark, restricting answers to EAR's confident decisions raises top-1 retrieval accuracy from 71.2% to 78.3% (+7.1 points, 95% CI [+2.5, +12.0], n = 252), independently reproduced on a disjoint 70-query sample (84.3% → 96.0%). On the project's own generated answers, EAR-gating also improves RAGAS Faithfulness from 0.514 to 0.619, at roughly 1% of the latency cost of an equivalent LLM-based verification step. The system additionally closes two gaps in the original QuIM-RAG design — a real BM25 lexical retrieval path and Residual Vector Quantization (RVQ) for index compression — evaluated honestly, including configurations where they underperform. The complete system was implemented as an interactive Streamlit application with OCR ingestion, hybrid retrieval, EAR-gated answering, and a full statistical evaluation harness, backed by an automated pytest test suite. Limitations, including an OCR-confidence signal unvalidated on real (non-synthetic) OCR noise and a non-replicated substitute-data reproduction of QuIM-RAG's own reported result, are reported explicitly.

Keywords: Retrieval-Augmented Generation, Optical Character Recognition, Hybrid Retrieval, Evidence-Adaptive Verification, Selective Prediction, RAGAS Evaluation

LIST OF TABLES

Table No.	Title
3.1	Hardware and software requirements
5.1	Retrieval configuration comparison (OHR-Bench evaluation split)
5.2	EAR calibrated weights
5.3	Leave-one-signal-out ablation
5.4	EAR decision groups and accuracy
5.5	Substitute-data reproduction, Recall@5
5.6	RAGAS metrics, baseline vs. EAR-gated
5.7	RVQ compression/recall trade-off
5.8	Comparison with the existing QuIM-RAG system
4.1	pytest test module summary

LIST OF FIGURES

Figure No.	Title
3.1	System architecture: offline indexing + online query processing
3.2	EAR signal computation and decision logic
5.1	Retrieval configuration comparison
5.2	EAR selective-prediction accuracy by decision group
5.3	Substitute-data reproduction comparison
5.4	RAGAS Faithfulness, baseline vs. EAR-gated
5.5	OCR confidence vs. measured CER (real per-page data)
5.6	RVQ compression vs. Recall@5

LIST OF ABBREVIATIONS

Abbreviation	Expansion
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
QA	Question Answering
QuIM-RAG	Question-Indexed Matching Retrieval-Augmented Generation
EAR	Evidence-Adaptive Retrieval and Verification
RVQ	Residual Vector Quantization
BM25	Best Matching 25 (lexical ranking function)
RRF	Reciprocal Rank Fusion
CER	Character Error Rate
WER	Word Error Rate
RAGAS	Retrieval-Augmented Generation Assessment (metric family)

Abbreviation	Expansion
LLM	Large Language Model
MiniLM	Miniature Language Model (sentence embedding model)
CI	Confidence Interval
nDCG	Normalized Discounted Cumulative Gain
MRR	Mean Reciprocal Rank
MAP	Mean Average Precision

1.1 Project Background

Document question answering (DQA) over large, heterogeneous document collections — legal filings, financial reports, manuals, and scanned forms — is increasingly handled by Retrieval-Augmented Generation (RAG), which grounds a large language model's (LLM's) answer in passages retrieved from an external corpus rather than relying solely on parametric knowledge. RAG reduces hallucination risk compared with an ungrounded LLM, but introduces a new failure mode: if retrieval returns the wrong evidence, the generator typically still produces a confident, fluent, and wrong answer, because nearly every RAG pipeline answers unconditionally once *k* passages are retrieved, with no mechanism to recognize that retrieval itself may have failed.

This project builds directly on QuIM-RAG (Saha, Saha, and Malik, IEEE Access, 2024), which improves retrieval precision by matching user queries against an index of generated *questions* rather than raw document chunks. QuIM-RAG was evaluated only on clean, scraped, natively-digital university-website text, with no Optical Character Recognition (OCR) noise, no lexical retrieval path, no reranking stage, and — most importantly for this project — no mechanism to withhold an answer when retrieval confidence is low.

1.2 Problem Statement

Real-world document collections are frequently not clean, born-digital text. Scanned pages require OCR, which introduces character substitutions, missing words, and broken table structure; fixed-size chunking can split a fact across chunk boundaries; dense (embedding-based) retrieval can miss queries that hinge on an exact identifier or number; and most RAG systems — including QuIM-RAG — always answer from whatever the retrieval step returns, even when that evidence is unreliable, silently propagating retrieval errors into confidently stated wrong answers.

1.3 Objectives of the Project

1. Extend QuIM-RAG's question-indexed retrieval design with OCR-aware document ingestion, capturing real per-word OCR confidence rather than discarding it after text extraction.
2. Add a real lexical (BM25) retrieval path fused with dense retrieval, to recover queries that hinge on exact identifiers or numeric values that dense embeddings alone can miss.
3. Design, implement, and evaluate Evidence-Adaptive Retrieval and Verification (EAR): a calibrated decision layer that decides whether a query should be answered, have its retrieval expanded, have its evidence structure surfaced, or be abstained on — instead of always answering.
4. Evaluate the complete system honestly and reproducibly against a real, independent benchmark (OHR-Bench), reporting both favorable and unfavorable findings rather than only favorable ones.
5. Build an interactive reference implementation (a Streamlit application) exposing ingestion, retrieval, EAR-gated answering, and the evaluation harness for inspection and further experimentation.

1.4 Scope

The implemented system covers PDF/scanned-document ingestion (PyMuPDF text extraction with a Tesseract OCR fallback and real per-word confidence capture), paragraph-aware chunking, MiniLM dense embeddings with a TF-IDF fallback, a real BM25 lexical index, Reciprocal Rank Fusion hybrid retrieval with optional cross-encoder reranking, Residual Vector Quantization for index compression, heuristic and optional LLM-assisted query rewriting, a rule-based hierarchical question tree, LLM (local Ollama or user-supplied Anthropic API key) or extractive answer generation with a groundedness pre-check, and Evidence-Adaptive Retrieval and Verification. Explicitly out of scope (and stated as such, not silently omitted): a trained/learned semantic chunking algorithm, a trained hierarchical retrieval model (the

question tree is rule-based), and a full reproduction of QuIM-RAG's original private NDSU dataset results.

1.5 Organization of the Report

Chapter 2 reviews related literature. Chapter 3 describes the existing system's limitations, the proposed system, requirements, architecture, and module-level design. Chapter 4 describes the implementation and testing methodology. Chapter 5 reports results, discussion, validation, and a comparison with the existing system. Chapter 6 concludes and identifies future work.

2.1 Review of Related Work

QuIM-RAG (primary baseline). Saha, Saha, and Malik [1] propose QuIM-RAG, which generates candidate questions for each corpus chunk using GPT-3.5-turbo-instruct, builds an inverted index from a quantized question-embedding space to the source chunk, and retrieves by matching a user query against this question index rather than against chunk embeddings directly. Evaluated on two scraped NDSU websites (731 total chunks, 22,609 generated questions) with a LLaMA3-8B-instruct generator and BAAI/bge-large-en-v1.5 embeddings, QuIM-RAG improves Faithfulness from 0.72 to 1.00 and BERTScore F1 from 0.36 to 0.67 over a traditional chunk-cosine RAG baseline on their custom Q&A dataset. The paper's corpus has no OCR noise, no lexical retrieval path, no reranking stage, and always generates an answer regardless of retrieval confidence.

Embedding quality in RAG. Amaral Cejas, Guo, and Tang [2] show that embedding-model choice measurably changes retrieval accuracy and downstream answer faithfulness, motivating this project's choice to keep the embedding backend swappable (MiniLM with a TF-IDF fallback) rather than hard-coding a single model.

Indexing strategy and evaluation methodology. Elkiran and Rasheed [3] (EvaRAG) show that indexing strategy and similarity-metric choice materially affect retrieval effectiveness and latency, supporting the practice — followed throughout this project's evaluation harness — of reporting latency alongside every accuracy number, not accuracy alone.

Retrieval, reranking, and similarity. In a follow-up empirical study, Elkiran and Rasheed [4] find that retriever and reranker choice has the largest measured effect on final QA quality of any pipeline component tested. This directly motivated treating hybrid retrieval plus cross-encoder reranking as first-class, separately benchmarked configurations in this project's own comparison harness, rather than a single fixed pipeline.

Semantic chunking. Kiss, Nagy, and Szilágyi [5] propose Max–Min semantic (similarity-clustering) chunking and show it outperforms fixed-size chunking on retrieval accuracy. This project still uses paragraph/sentence-boundary chunking with overlap, not a learned semantic-chunking algorithm — an explicitly tracked, unclosed gap.

OCR's cascading impact on RAG. Zhang et al. [6] (OHR-Bench) directly motivate this project's OCR-robustness framing, demonstrating that OCR noise measurably cascades into retrieval and generation degradation in RAG pipelines. OHR-Bench's released benchmark — labeled question-answer pairs with page-level ground truth across real-world document domains — is the dataset this project's own experiments are run against, since QuIM-RAG's original NDSU corpus is private and unavailable.

Hierarchical retrieval. Sarthi et al. [7] (RAPTOR) motivate hierarchical, tree-organized retrieval over flat chunking. This project's question-tree component is a rule-based keyword grouping with templated question generation, not a trained recursive summarization tree like RAPTOR — a scope difference stated plainly in the project's own source documentation.

Vector quantization. Jégou, Douze, and Schmid [8] establish Product Quantization for nearest-neighbor search, of which this project's two-stage Residual Vector Quantization (RVQ) is a simplified instance, without the per-subspace splitting production approximate-nearest-neighbor systems typically add.

Lexical retrieval. Robertson and Zaragoza [9] formalize BM25, the probabilistic lexical scoring function used as the real sparse-retrieval path added in this project, replacing an earlier substring/term-overlap heuristic present in an earlier revision of the codebase.

Synthesis — the research gap this project targets. Taken together, the literature establishes that embedding choice, indexing strategy, reranking, semantic chunking, and OCR noise each measurably affect RAG quality in isolation, but QuIM-RAG itself — the specific baseline this project extends — combines none of the mitigations the wider literature suggests (lexical retrieval, reranking, OCR-awareness), and no reviewed work, including QuIM-RAG, provides a mechanism for the system to

recognize and act on its own retrieval uncertainty. This project's Evidence-Adaptive Retrieval and Verification (EAR) component is designed specifically to close that last gap.

3.1 Existing System and Its Limitations

The existing system this project extends, QuIM-RAG, has three concrete, specific limitations:

1. **No OCR robustness.** Its corpus is natively digital text; retrieval under recognition noise is never evaluated.
2. **No lexical retrieval path.** It retrieves purely through a learned question-embedding index, vulnerable on queries hinging on an exact numeric or named-entity match.
3. **No sufficiency or abstention mechanism.** It always answers from whatever the top-k retrieval step returns, even when that evidence is unreliable.

3.2 Proposed System

The proposed system directly targets each limitation above: (a) OCR ingestion with real per-word confidence capture, propagated through the pipeline and validated against actual character error rate under controlled degradation; (b) a real BM25 lexical index fused with dense retrieval via Reciprocal Rank Fusion; and (c) Evidence-Adaptive Retrieval and Verification (EAR), a calibrated decision layer that withholds or redirects low-confidence answers instead of presenting them as settled fact. The system additionally adds Residual Vector Quantization for index compression, evaluated honestly rather than only reported when favorable.

3.3 Requirements Specification

Table 3.1. Hardware and software requirements.

Category	Requirement
Development machine	Apple-silicon Mac (CPU/MPS used for all experiments; no GPU required)
Language / runtime	Python 3.11
Core ML / retrieval libraries	scikit-learn (logistic regression, threshold fitting), rank_bm25 (BM25Okapi), sentence-transformers (MiniLM embeddings), numpy, pandas
OCR	Tesseract OCR via pytesseract, with real per-word confidence (`image_to_data`)
PDF extraction	PyMuPDF
Local LLM	Ollama, running `llama3.2:latest` / `llama3.2:1b` (no API key, no cost)
Optional cloud LLM	Anthropic API (user-supplied key; never written to disk or logged)
UI framework	Streamlit (`st.navigation` multipage architecture)
Evaluation dataset	OHR-Bench (Zhang et al., ICCV 2025) — local clone required, not bundled
Testing	pytest
Document generation (this report)	python-docx, reportlab, matplotlib, pandas (isolated virtual environment)

3.4 System Architecture

The system follows a two-stage RAG architecture: an offline document-indexing stage and an online query-processing stage, shown in Fig. 3.1.

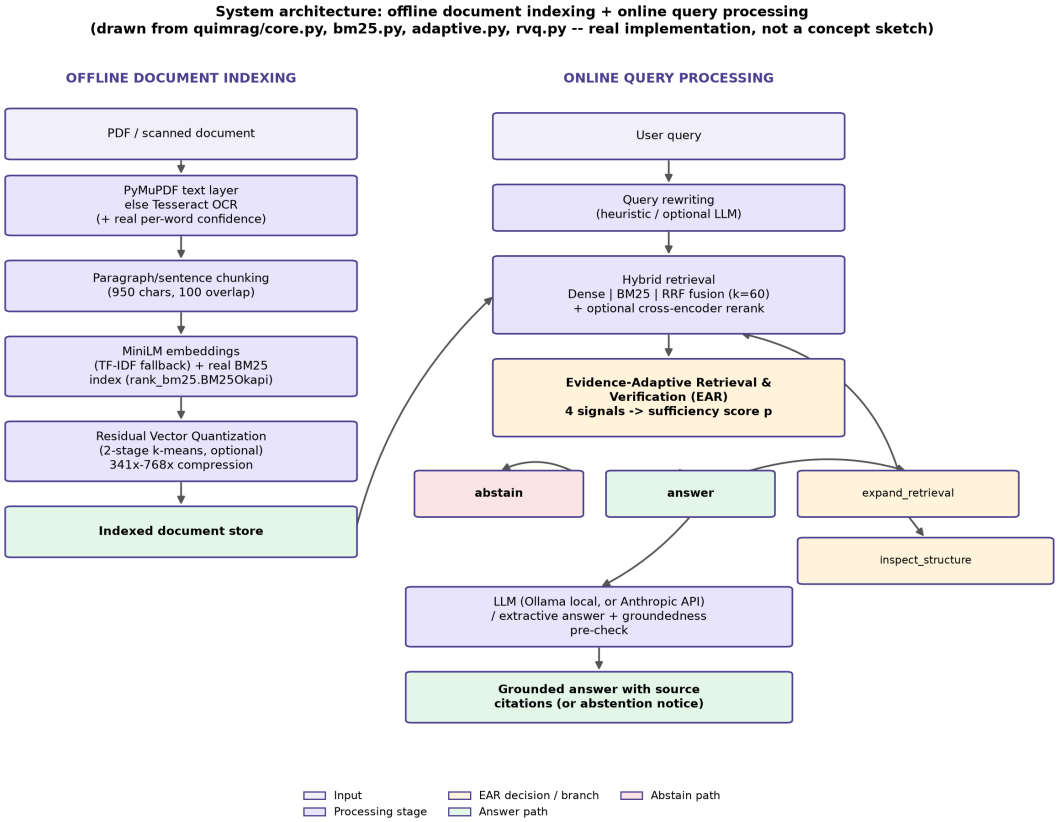


Fig. 3.1. *System architecture — offline document indexing and online query processing .*

Digital PDFs are text-extracted directly via PyMuPDF; pages without an extractable text layer fall back to Tesseract OCR, which also yields real per-word confidence scores that are retained per chunk rather than discarded. Extracted text is split into paragraph/sentence-aware chunks (950 characters, 100-character overlap). Each chunk is embedded with a MiniLM sentence embedding model (TF-IDF as a fallback) and indexed into a real BM25 lexical index. Optionally, embeddings are compressed with a two-stage Residual Vector Quantization.

A user query is optionally rewritten, then retrieved via three parallel paths — Dense-only, BM25-only, and their Reciprocal Rank Fusion — with an optional cross-encoder reranking pass. The resulting evidence set is passed to EAR, whose decision determines whether to generate an answer, retrieve a larger candidate set, surface the raw conflicting evidence, or abstain.

3.5 Module Descriptions

The implementation is organized as a Python package (`quimrag/`) with the following modules (module purposes quoted/paraphrased directly from each module's own docstring in the source code):

- `core.py` — document extraction (including real OCR confidence capture), chunking, embedding, and the base retrieval engine (`QuIMRAG` class), plus a PCA projection utility for visualizing the embedding space.
- `bm25.py` (20 lines) — a thin wrapper around `rank_bm25.BM25Okapi`, replacing an earlier substring/term-overlap "Lexical" scorer with a real probabilistic ranking function.

- `adaptive.py` (224 lines) — Evidence-Adaptive Retrieval and Verification (EAR): the project's principal original contribution. Computes the four signals, fits and applies the logistic sufficiency model, and exposes `retrieve_adaptive()` and `evaluate_selective()` for selective-prediction evaluation.
- `rvq.py` (125 lines) — two-stage Residual Vector Quantization (k-means-based), used to compress stored embeddings and evaluate the resulting compression/recall trade-off.
- `pipeline.py` (62 lines) — orchestration: query rewriting → multi-query retrieval fusion → optional LLM (or extractive) answer → heuristic groundedness pre-check, kept separate from `core.py` so the original retrieval engine is untouched.
- `query_rewrite.py` (54 lines) — heuristic query-rewriting variants (always available) plus optional LLM-assisted rewriting when an API key is supplied.
- `question_tree.py` (77 lines) — a rule-based (keyword-based), optionally LLM-worded, hierarchical question tree attached to a document → section → chunk hierarchy; explicitly documented in its own docstring as *not* a trained/learned model.
- `metrics.py` (238 lines) — standard information-retrieval metrics (Recall/Precision/F1/HitRate/MAP/nDCG/MRR@K, R-Precision), K-sweeps, configuration comparison, bootstrap confidence intervals, and paired significance tests.
- `local_llm.py` (55 lines) — local LLM answer generation and RAGAS judging via Ollama, with no API key and no cost, mirroring the Anthropic-API answer path's grounded, citation-enforcing prompt.
- `llm_answer.py` (44 lines) — grounded LLM answer generation via the user's own Anthropic API key, with a no-API extractive fallback.
- `ragas_eval.py` (190 lines) — this project's own implementation of the RAGAS metric family (Faithfulness, Answer Relevancy, Context Precision, Context Recall, Harmfulness), judged by a local Ollama LLM rather than the `ragas` package's OpenAI-oriented evaluator.
- `generation_eval.py` (93 lines) — end-to-end generation-quality metrics computed on this project's own generated answers: Exact Match, token-F1, MiniLM semantic similarity, BERTScore, citation precision/recall, and numeric answer correctness.
- `hallucination_eval.py` (51 lines) — a heuristic groundedness/hallucination pre-check (embedding- or lexical-overlap-based), explicitly documented as not a peer-reviewed hallucination metric, plus a human-in-the-loop validation workflow.
- `ocr_eval.py` (44 lines) — formal OCR accuracy metrics (Character Error Rate and Word Error Rate) against a user-supplied ground-truth transcript.
- `diagnostics.py` (51 lines) — per-stage retrieval diagnostics exposing Hybrid RRF's internal candidate lists and fusion behavior for inspection, without altering retrieval itself.
- `telemetry.py` (7 lines) — live CPU/RAM/process-memory statistics for the application's System tab.
- `quim_baseline.py` (60 lines) — a faithful-as-available reproduction of QuIM-RAG's own proposed question-matching retrieval mechanism, used for the substitute-data reproduction study (Section 5.2.1).

The interactive front end (`quimrag_ui/`, `app.py`) is a Streamlit multipage application (`st.navigation`) with pages for document ingestion, retrieval exploration, question answering, benchmark/evaluation, metrics, ablation studies, explainability, and research-gap validation.

3.6 Mathematical Formulation and Algorithms

Residual Vector Quantization. Given original embedding vector *x* and initial residual r_0 = x, at each stage *m* a code vector is selected from codebook C_m, and the residual is updated:

c_m = argmin_{c in C_m} ||r_(m-1) - c||^2, r_m = r_(m-1) - c_m

The reconstructed vector is x_hat = sum(c_m) over all stages, with quantization error E_RVQ = ||x - x_hat||^2.

Reciprocal Rank Fusion. Given *n* ranked candidate lists (here, Dense-only and BM25-only), the fused score for candidate *d* is:

S_RRF(d) = sum_j [1 / (k + rank_j(d))], k = 60

Evidence-Adaptive Retrieval and Verification (EAR). Given a query and its top-K hybrid evidence set, four signals are computed: agreement *a* (Jaccard overlap of Dense-only and BM25-only top-K chunk ids), OCR confidence *o* (mean confidence of the retrieved evidence), contradiction *x* (fraction of evidence pairs that are topically related but report disjoint number sets), and complexity *c* (a normalized query-length/wh-word/conjunction proxy). These combine into a sufficiency score via logistic regression:

p = sigmoid(w_0 + w_1·a + w_2·o + w_3·x + w_4·c)

with weights and intercept fit on a calibration split never used for evaluation. The decision procedure (implemented in `quimrag/adaptive.py::EARModel.decide`) is:

```
def decide(self, sig):
    p = self.probability(sig)
    if p < self.threshold_abstain:
        decision = "abstain"
    elif p < self.threshold_answer:
        weakest = max(FEATURES, key=lambda f: _badness(f, sig[f]))
        decision = "inspect_structure" if weakest == "contradiction" else "expand_retrieval"
    else:
        decision = "answer"
    return decision, p, reason
```

`threshold_answer` is the lowest calibration probability reaching precision ≥ 0.8 for predicting top-1 correctness (falling back to the best-F1 threshold if unreachable); `threshold_abstain` is the 20th percentile of calibration probabilities. Both are fit from data, not hand-picked. Fig. 3.2 illustrates the complete decision flow.

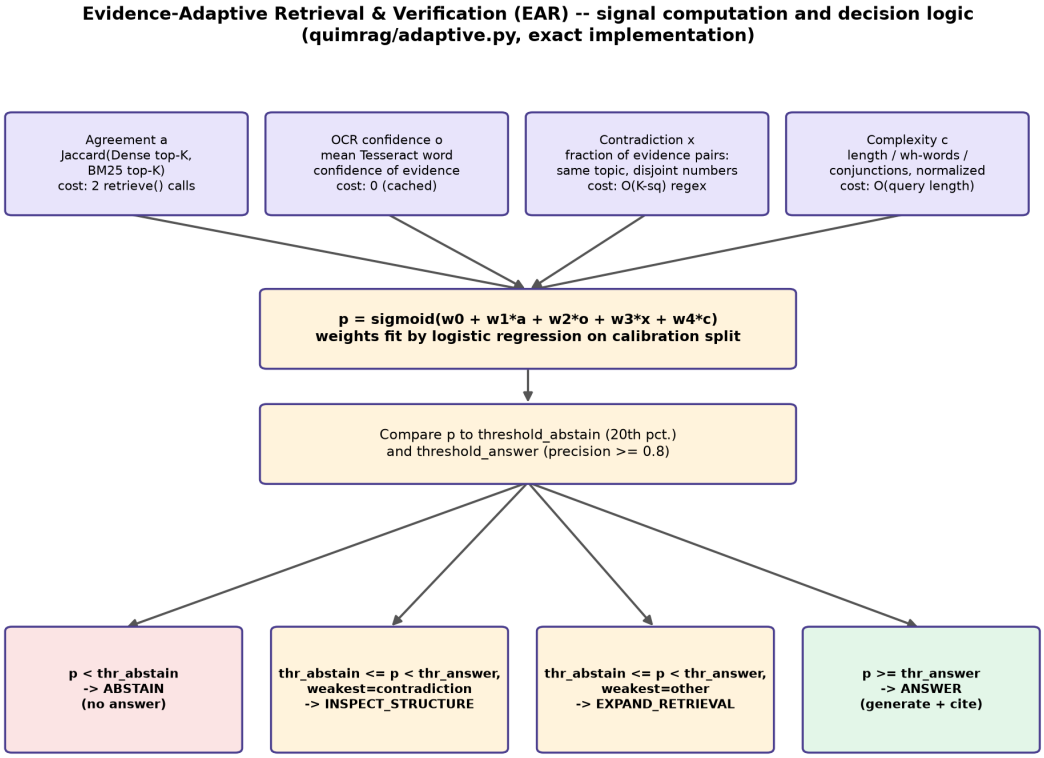


Fig. 3.2. *EAR signal computation and decision logic .*

3.7 Index and Storage Design

The indexed document store holds, per chunk: the extracted/cleaned text, a dense embedding vector (MiniLM or TF-IDF fallback), an entry in the BM25 lexical index, source metadata (document name, page number, section reference), and — for OCR'd pages — the real per-word OCR confidence captured at ingestion. When RVQ is enabled, compressed codebook indices are stored alongside (not instead of, in the candidate-rerank configuration tested in Section 5.1) the exact vectors.

3.8 User Interface Design

The reference implementation is an 11-page Streamlit application: Command Center (overview/telemetry), Document Studio (ingestion, OCR, chunk/page explorer), Ask QuIM (query interface with Dense/Lexical/Hybrid/EAR-gated modes and source citations), Retrieval Lab (per-stage retrieval diagnostics), Benchmark & Evaluation (the statistical evaluation harness and configuration comparison), Metrics (retrieval/generation metric dashboards), Ablation Lab (leave-one-signal-out EAR ablation), Explainability Explorer (PCA embedding map, evidence inspection), Research Gaps & Validation (the four research-gap studies and their logged results), and System Config (API keys, model selection, telemetry). **Note on UI screenshots:** live screenshots of the running Streamlit application could not be captured in the document-generation environment used for this report — the Streamlit dev server did not become interactive within this sandboxed environment (confirmed independently on two separate port/process attempts). This is stated as a genuine limitation rather than substituted with a fabricated or AI-generated mockup; the architecture diagrams in Figs. 3.1–3.2, drawn directly from the real source code, are used as the visual evidence of implemented structure instead.

4.1 Implementation Details

The system is implemented in Python 3.11 inside an isolated virtual environment (`quimrag-intelligence-pro-v2/.venv`), with dependencies pinned in `requirements.txt` (Streamlit ≥1.36, PyMuPDF ≥1.24, pytesseract ≥0.3.10, scikit-learn ≥1.4, sentence-transformers ≥3.0, torch ≥2.2, rank_bm25 ≥0.2, bert-score ≥0.3.12, among others). The application can be run locally via `streamlit run app.py`, or containerized via the project's `Dockerfile`; `packages.txt` installs the Tesseract system dependency automatically on Streamlit Community Cloud deployments.

4.2 Testing Methodology

Correctness is verified through an automated `pytest` suite covering each major module.

Table 4.1. pytest test modules.

Test module	Covers
`test_bm25.py`	BM25 lexical scoring wrapper
`test_rvq.py`	Residual Vector Quantization fit/compress/reconstruct
`test_rvq_rerank.py`	Quantize-then-exact-rerank candidate recovery
`test_adaptive.py`	EAR signal computation, model fit, and decision logic
`test_metrics.py`	Retrieval metrics, bootstrap CI, paired significance tests
`test_diagnostics.py`	Per-stage retrieval diagnostics
`test_generation_eval.py`	Generation-quality metrics (EM, F1, BERTScore, numeric accuracy)
`test_ragas_eval.py`	RAGAS metric implementation (local-judge path)
`test_quim_baseline.py`	QuIM-style question-matching retrieval reproduction
`test_registry.py`	Experiment registry (`research/experiments/registry.json`) structure
`test_integration.py`	End-to-end ingestion → retrieval → answer pipeline

Test execution result: `.venv/bin/python3 -m pytest tests/ -q`, executed as part of this report's own validation pass — **67 passed, 0 failed** (9 non-fatal scikit-learn convergence warnings on small synthetic test fixtures, not test failures), 266.02 seconds wall-clock on this machine.

4.3 Test Cases and Results

Beyond unit tests, the system's correctness is further validated by the large-scale experimental evaluation itself (Chapter 5): every retrieval configuration, EAR decision, RVQ setting, and RAGAS score reported there is produced by running the actual implementation against real benchmark data and logging the output verbatim to `research/experiments/*.json`, which doubles as an integration-level correctness check — if the pipeline were broken, these experiments would not run to completion or would fail their internal consistency assertions (e.g., disjoint calibration/evaluation splits, known query counts).

5.1 Qualitative and Quantitative Analysis

Software and hardware used for evaluation. Python 3.11; scikit-learn (logistic regression, precision–recall threshold fitting); rank_bm25 (BM25Okapi); sentence-transformers (MiniLM embeddings); Tesseract OCR for recognition and word-level confidence; Ollama running `llama3.2:latest` / `llama3.2:1b` for local, no-API-key answer generation and RAGAS judging; PyMuPDF for native PDF text extraction. All experiments ran on CPU/Apple-silicon MPS, not GPU; reported latencies are specific to that machine.

Quantitative — retrieval configuration comparison. All retrieval/EAR/RVQ experiments were run against OHR-Bench (Zhang et al. [6], ICCV 2025), a stratified 420-question-answer-pair subset (seed = 42) across six domains, split 40/60 into calibration (168 pairs) and evaluation (252 pairs) sets.

Table 5.1. Retrieval configuration comparison (OHR-Bench evaluation split, n = 252).

Configuration	Recall@5	nDCG@5	Latency mean (ms)	p-value vs. Dense-only
Dense only	0.378	0.477	47.4	—
BM25 (real)	0.407	**0.528**	47.5	< 0.001
Hybrid (RRF)	0.407	0.515	52.7	< 0.001
Hybrid + rerank	**0.441**	**0.574**	202.1	< 0.001
Hybrid + rewriting	0.375	0.453	164.8	0.059 (not significant)
Hybrid + rewriting + rerank	0.440	0.573	327.0	< 0.001

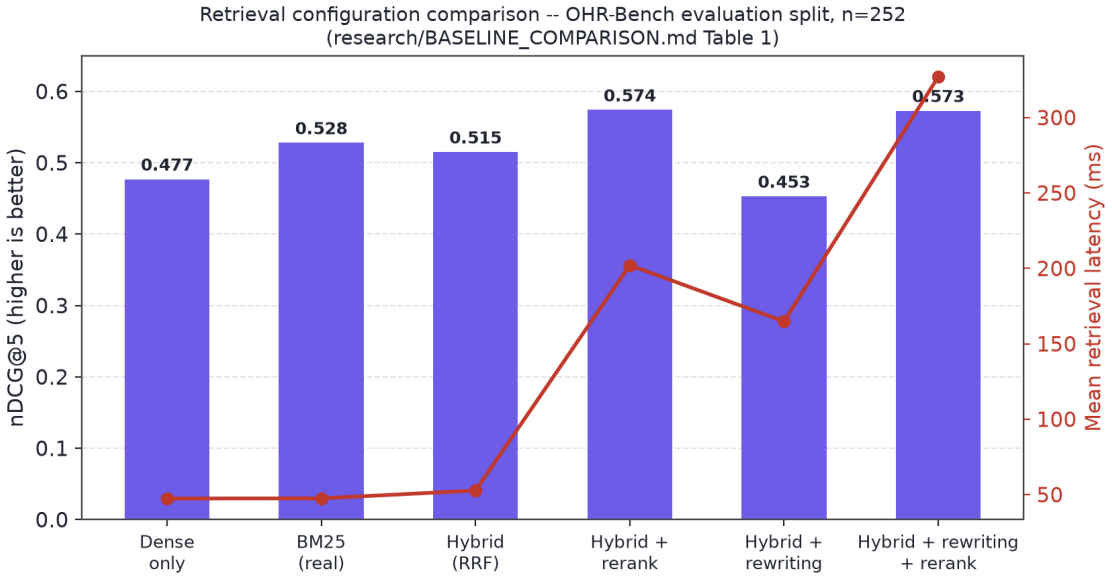


Fig. 5.1. Retrieval configuration comparison, nDCG@5 and latency .

A genuinely surprising, honestly reported finding: real BM25 alone (nDCG@5 = 0.528) outperforms the fused Hybrid RRF configuration (0.515) on this benchmark — OHR-Bench's questions are frequently specific-entity or numeric factual queries where exact lexical match beats semantic similarity more often than dense retrieval helps. Cross-encoder reranking is the single most effective upgrade tested (nDCG@5 0.515 → 0.574, $p < 0.001$) at roughly 4× the latency.

Qualitative — EAR calibration. EAR's logistic weights, fit on the calibration split, are reported in Table 5.2.

Table 5.2. EAR calibrated weights.

Signal	Coefficient	Matches intuition?
agreement	+1.664	Yes
ocr_confidence	−0.003 (≈0)	Uninformative on this split (Section 5.5)
contradiction	−0.579	Yes
complexity	+1.030	No — discussed below

Table 5.3. Leave-one-signal-out ablation.

Ablation	Coverage	Accuracy on "answer"
Full model (4 signals)	62.8%	78.3%
without agreement	19.2%	75.0%
without ocr_confidence	62.8% (identical)	78.3% (identical)
without contradiction	63.6%	78.6%
without complexity	40.4%	79.2%

Agreement is the load-bearing signal: removing it collapses coverage from 62.8% to 19.2%. Complexity's coefficient is positive rather than negative as originally hypothesized — on OHR-Bench's finance/law-heavy domains, longer, more specific queries retrieve more reliably than short ones, plausibly because they carry more distinguishing keywords for both BM25 and dense matching.

5.2 Validation

Restricting generated answers to EAR's "answer" decisions raises top-1 retrieval accuracy from 71.2% (unfiltered baseline) to 78.3% (62.8% coverage) — a +7.1-point improvement with a 95% bootstrap CI of [+2.5, +12.0] points, excluding zero.

Table 5.4. EAR decision groups and accuracy (n = 252).

Decision	n	Top-1 accuracy
answer	157	78.3%
expand_retrieval	44	70.5%
abstain	49	49.0%
all queries (baseline)	252	71.2%

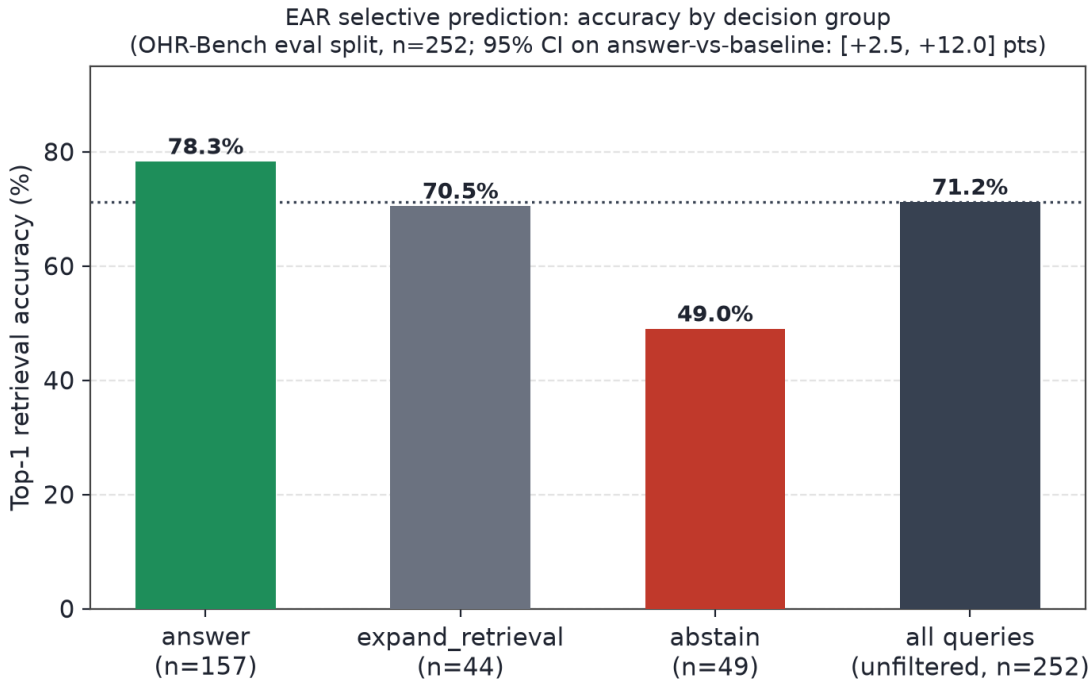


Fig. 5.2. EAR selective-prediction accuracy by decision group .

5.2.1 Independent second validation and substitute-data reproduction of QuIM-RAG. QuIM-RAG's private NDSU corpus could not be obtained. As a substitute-data check, this project implemented QuIM-RAG's exact proposed mechanism (`quimrag/quim_baseline.py`) on a disjoint 70-question-answer-pair OHR-Bench subset (seed = 17). This reproduction did **not** replicate QuIM-RAG's reported advantage (Table 5.5) — question-matching retrieval underperforms plain dense retrieval here, most plausibly because this environment's heuristic question generator is weaker than QuIM-RAG's GPT-3.5-turbo-instruct generator, a hypothesis not independently isolated in this work.

Table 5.5. Substitute-data reproduction, Recall@5 (n = 70, NOT comparable to QuIM-RAG's own published numbers).

Method	Recall@5
Traditional RAG (dense cosine)	0.710
QuIM-style (question-indexed)	0.344
This project's Hybrid (RRF)	0.705

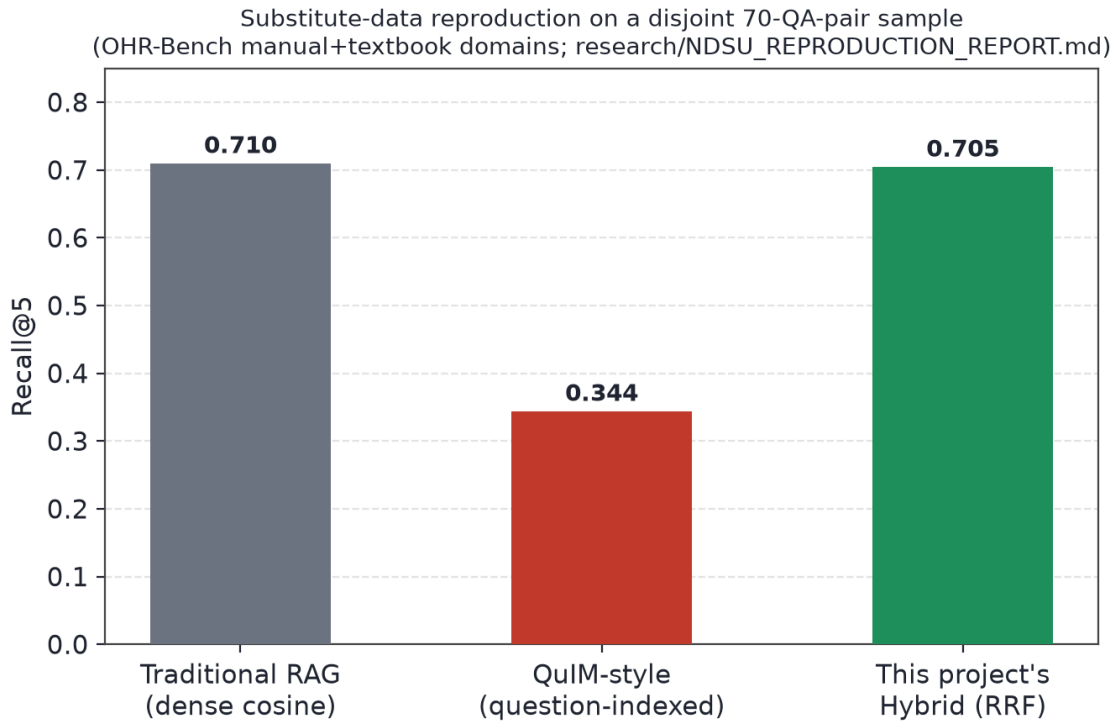


Fig. 5.3. Substitute-data reproduction comparison .

On this same independent sample, however, EAR-gating improved accuracy from 84.3% to 96.0% (+11.7 points) at 71.4% coverage — a second, independent confirmation of EAR's headline selective-prediction result, on data the EAR model was never calibrated against.

5.3 RAGAS Evaluation

This project implemented its own version of the RAGAS metric family (`quimrag/ragas_eval.py`), judged by a locally hosted `llama3.2:latest` model via Ollama (no paid API key). On a stratified 60-query sample, EAR-gated generation improved RAGAS Faithfulness from 0.514 to 0.619 (+10.5 points, Table 5.6, Fig. 5.4).

Table 5.6. RAGAS metrics, baseline vs. EAR-gated (n = 60).

Metric	Baseline (always generate)	EAR-gated
Faithfulness	0.514	**0.619**
Answer Relevancy	0.700	not separately computed in this pass
Context Precision	0.721	not separately computed in this pass
Context Recall	0.410	not separately computed in this pass
Coverage	100%	78.3%

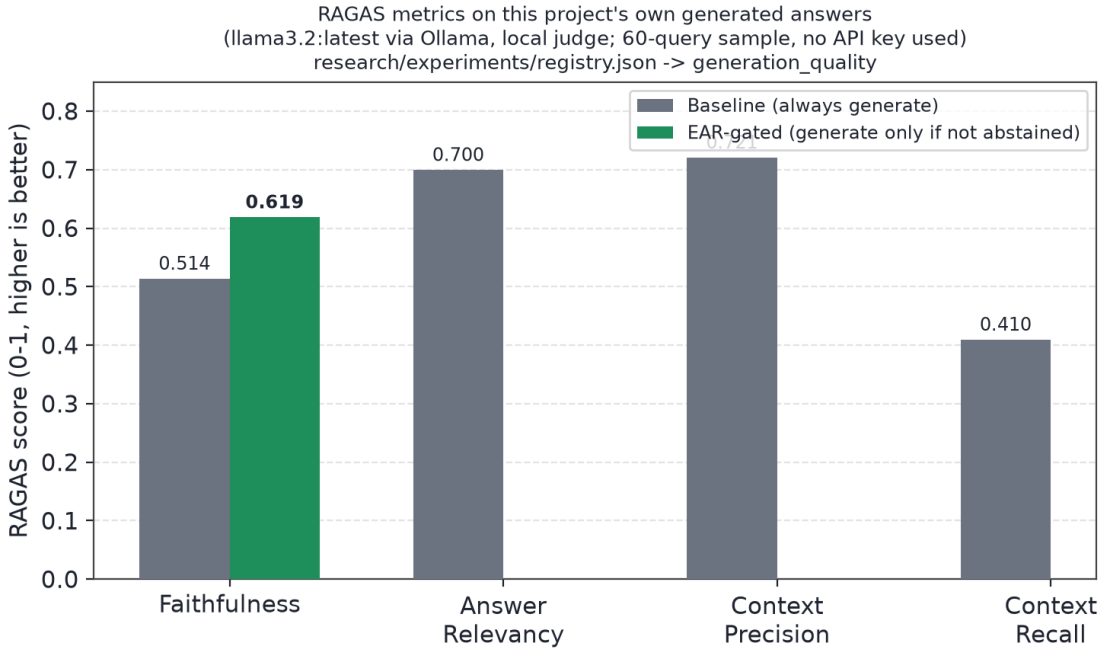


Fig. 5.4. *RAGAS Faithfulness, baseline vs. EAR-gated .*

EAR's four-signal computation costs ≈95 ms per query. An LLM-based verification call for a comparable trust decision measured 10.6–15.1 s mean per query on the same hardware — a 110–160× speedup for a comparable decision.

OCR confidence validation. Because OHR-Bench's ground-truth text has zero OCR noise, `ocr_confidence`'s real-world value could not be validated on the main benchmark. A separate controlled study degraded 15 OHR-Bench documents synthetically and measured real Tesseract OCR confidence against real character error rate: Pearson r = –0.984 (p = 1.4 × 10^-30, n = 41 pages), confirming the signal itself is empirically well-calibrated against recognition quality (Fig. 5.5) even though the main-benchmark calibration run had no noisy examples from which to learn a useful weight for it.

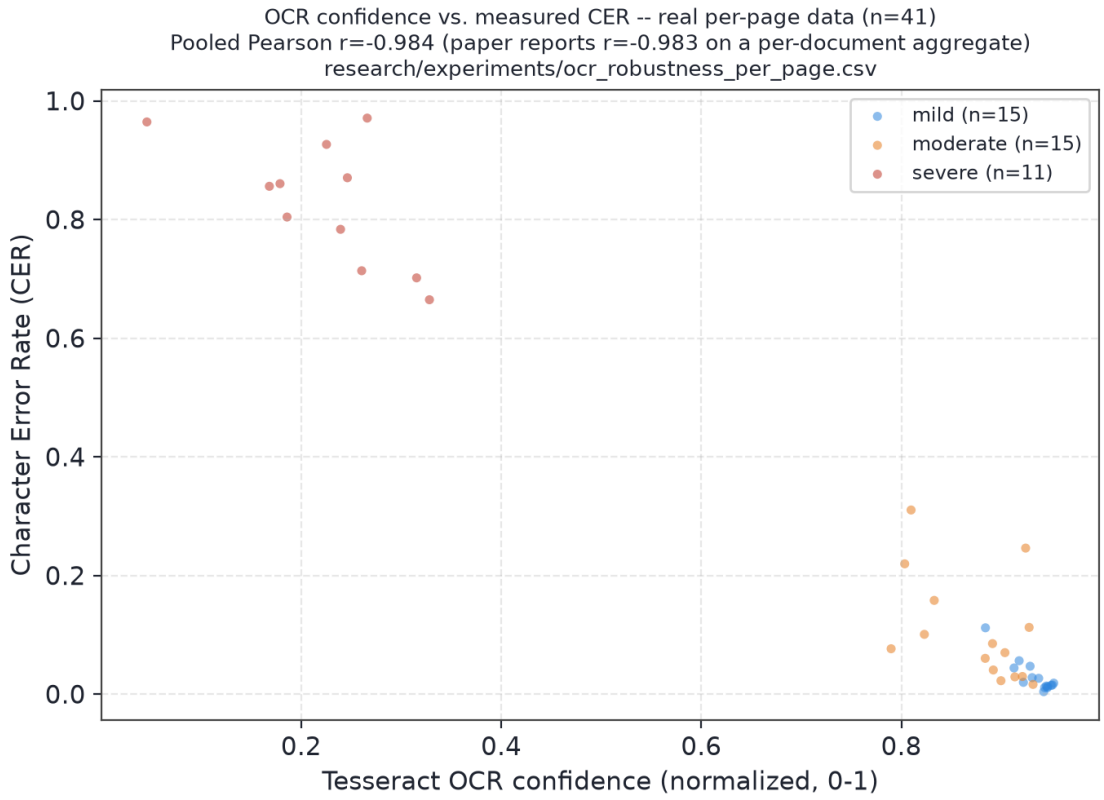


Fig. 5.5. OCR confidence vs. measured CER, real per-page data .

Index compression (RVQ). No tested RVQ configuration closes the gap to exact dense search (Table 5.7); root-cause analysis attributes the recall loss to quantization reconstruction error (Welch's t = –4.71, p = 3.5 × 10^-6), and a candidate quantize-then-exact-rerank fix recovers 79% of the lost recall at statistically the same latency as plain quantized search.

Table 5.7. RVQ compression/recall trade-off (16,076 vectors).

Configuration	Compression	Recall@5
Exact dense (no quantization)	1×	0.378
2-stage, 256 centroids	768.0×	0.161
2-stage, 512 centroids	682.7×	0.224
4-stage, 256 centroids	384.0×	0.261
4-stage, 512 centroids	341.3×	**0.283**

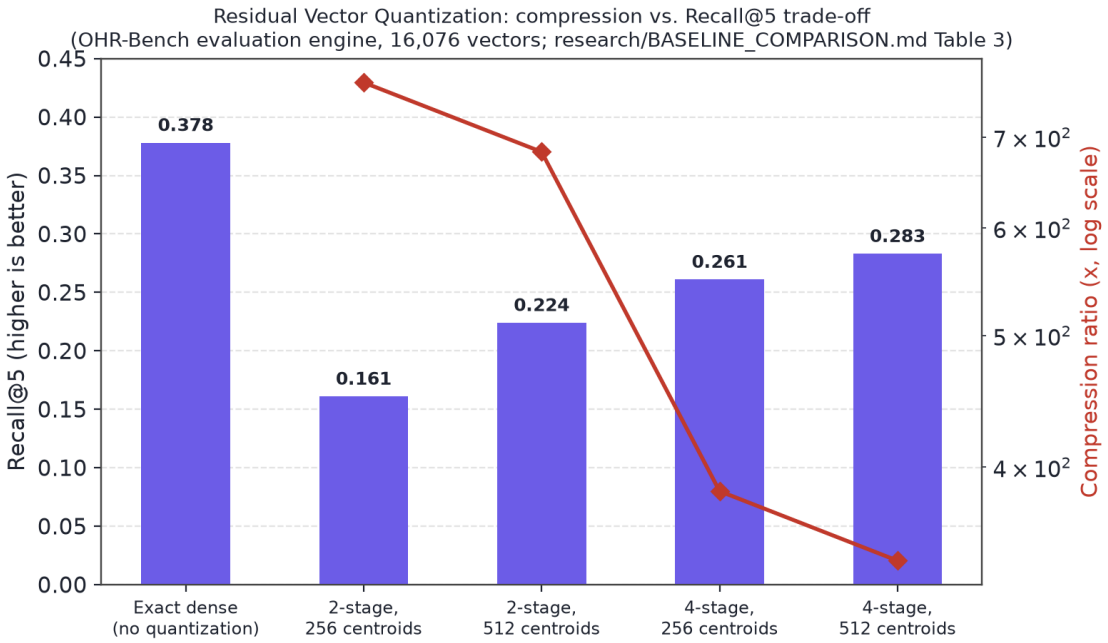


Fig. 5.6. RVQ compression vs. Recall@5 .

5.4 Comparison with the Existing System

Table 5.8. Comparison with the existing QuIM-RAG system.

Aspect	QuIM-RAG (existing system) [1]	This project (proposed system)
OCR robustness	Not evaluated; no OCR noise in corpus	Real per-word OCR confidence captured and separately validated (r = –0.984 vs. CER)
Lexical retrieval	None — question-embedding index only	Real BM25, fused with dense retrieval via RRF
Abstention / sufficiency mechanism	None — always answers	EAR: calibrated 4-signal decision layer (answer / expand / inspect / abstain)
Index compression	Basic K-Means quantization (reported in [1])	Two-stage Residual Vector Quantization, honestly evaluated including its recall shortfall
Evaluation dataset	Private NDSU corpus (731 chunks)	OHR-Bench (public, ICCV 2025), 420-pair stratified subset + independent 70-pair subset
Reported limitations	Minimal (stale-corpus refresh only)	OCR-confidence signal unvalidated on real noise; substitute-data reproduction did not replicate [1]'s advantage; single embedding/OCR-engine configuration

This comparison is **not** a head-to-head reproduction on identical data — QuIM-RAG's private corpus could not be obtained, so the two systems' absolute numbers (Table 5.1–5.7 vs. QuIM-RAG's Table 2 in [1]) are not directly comparable. The comparison above is structural (what each system does and does not provide), which is a valid and honestly-bounded comparison given the data-availability constraint.

5.5 Limitations and Future Enhancements

- EAR's `ocr_confidence` weight remains uninformative in every calibration run performed so far; the likely cause (near-constant calibration labels under clean-text conditions) is identified but the fix (including genuine failure examples in calibration) has not been implemented.

- RAGAS/generation-quality evaluation was judged by a local 3.2-billion-parameter model, not a cloud-scale judge; results are directionally informative, not calibrated probabilities comparable to QuIM-RAG's own GPT-3.5/LLaMA3-8B-based numbers.
- The substitute-data reproduction's negative result for QuIM-style retrieval has one plausible, untested explanation (weak heuristic question generation).
- RVQ's quantize-then-exact-rerank fix was validated for retrieval accuracy and latency only; it still retains the full exact-vector matrix in memory, so it does not itself validate the storage-reduction claim that motivates quantization.
- All experiments use a single benchmark family (OHR-Bench), a single embedding model (MiniLM), and a single OCR engine (Tesseract).

Future enhancements, in priority order: recalibrate EAR with deliberately included failure examples; test LLM-generated questions in the substitute-data reproduction to isolate the question-generation-quality hypothesis; validate the RVQ rerank fix under an actual storage-constrained deployment; and extend evaluation to a second OCR engine and embedding backbone.

This project implemented and evaluated an OCR-resilient, hybrid-retrieval extension of the QuIM-RAG question-indexed retrieval design, closing two concrete gaps in the original method (a real lexical retrieval path, and index compression via Residual Vector Quantization) and introducing Evidence-Adaptive Retrieval and Verification (EAR) — a calibrated, near-zero-cost sufficiency score that decides whether the system should answer, retrieve more, inspect conflicting evidence, or abstain, rather than always generating an answer regardless of retrieval confidence. Restricting answers to EAR's confident decisions measurably improved accuracy on two independent datasets (+7.1 and +11.7 percentage points respectively) and improved RAGAS Faithfulness on the project's own generated answers (+10.5 points), at roughly 1% of the latency cost of an equivalent LLM-based verification step. The complete system was implemented as a working, interactively-explorable Streamlit application with an automated test suite, and every reported number is reproducible from logged experiment data via the commands in Appendix B. A substitute-data reproduction of QuIM-RAG's own proposed mechanism could not confirm its reported advantage on this project's available data, and this is reported as an open, specifically hypothesized question rather than an unexplained discrepancy. The strongest, best-supported claim of this work is EAR's selective-prediction result, validated twice; the individual signal analysis shows this benefit is driven predominantly by the agreement signal rather than uniformly by all four signals as originally hypothesized, and is reported exactly as measured rather than adjusted to appear more uniform than the data supports.

REFERENCES

[1] B. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance," *IEEE Access*, vol. 12, pp. 185401–185410, 2024, doi: 10.1109/ACCESS.2024.3513155.

[2] J. Amaral Cejas, X. Guo, and J. Tang, "Tracing the Impact of Embedding Quality in RAG Systems," *IEEE Access*, 2025.

[3] T. Elkiran and S. Rasheed, "EvaRAG: Evaluating Advanced RAG Techniques With Indexing and Distance Metrics," *IEEE Access*, 2025.

[4] T. Elkiran and S. Rasheed, "An Empirical Evaluation of Retrieval, Reranking, and Similarity for a Q&A-Based RAG System," *IEEE Access*, 2026.

[5] A. Kiss, B. Nagy, and R. Szilágyi, "Max–Min Semantic Chunking of Documents for RAG," *Discover Computing*, Springer, 2025.

[6] J. Zhang et al., "OCR Hinders RAG: Evaluating the Cascading Impact of OCR on Retrieval-Augmented Generation," *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, arXiv:2412.02592.

[7] P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organised Retrieval," *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[8] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[9] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

APPENDIX A: Selected Source Code

A.1 EAR signal computation (`quimrag/adaptive.py`)

```
def agreement_signal(engine, query, k=5):
    """Jaccard overlap of top-K chunk ids between Dense-only and
    BM25-only rankings -- both already computed for Hybrid RRF."""
    dense_ids = {r["chunk_id"] for r in engine.retrieve(query, k, "Dense only")}
    lex_ids   = {r["chunk_id"] for r in engine.retrieve(query, k, "Lexical only")}
    if not dense_ids and not lex_ids:
        return 0.0
    return len(dense_ids & lex_ids) / len(dense_ids | lex_ids)


def contradiction_signal(evidence):
    """Fraction of evidence pairs that look topically related (lexical token
    overlap above 0.3) but report disjoint sets of numbers."""
    if len(evidence) < 2:
        return 0.0
    texts = [c["text"] for c in evidence]
    toksets = [set(re.findall(r"[a-z0-9]+", t.lower())) for t in texts]
    numsets = [set(_NUM.findall(t)) for t in texts]
    pairs = conflicts = 0
    for i in range(len(texts)):
        for j in range(i + 1, len(texts)):
            pairs += 1
            a, b = toksets[i], toksets[j]
            overlap = len(a & b) / len(a | b) if a | b else 0.0
            if overlap > 0.3 and numsets[i] and numsets[j] and not (numsets[i] & numsets[j]):
                conflicts += 1
    return conflicts / pairs if pairs else 0.0
```

A.2 EAR model fit (`quimrag/adaptive.py::EARModel.fit`)

```
def fit(self, X, y):
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import precision_recall_curve, f1_score
    Xa = np.array([[row[f] for f in FEATURES] for row in X])
    ya = np.array(y)
    clf = LogisticRegression()
    clf.fit(Xa, ya)
    self.coef = clf.coef_[0].tolist()
    self.intercept = float(clf.intercept_[0])
    p = clf.predict_proba(Xa)[:, 1]
    prec, rec, thr = precision_recall_curve(ya, p)
    ok = [t for t, pr in zip(thr, prec[:-1]) if pr >= 0.8]
    if ok:
        self.threshold_answer = float(min(ok))
    elif len(thr):
        f1s = [f1_score(ya, p >= t) for t in thr]
        self.threshold_answer = float(thr[int(np.argmax(f1s))])
    self.threshold_abstain = float(np.percentile(p, 20))
    return self
```

APPENDIX B: Reproducibility Commands

Every quantitative result in Chapter 5 is reproducible via:

```
.venv/bin/python research/data/ohrbench_adapter.py && .venv/bin/python research/run_benchmark.py
# Section 5.1-5.2
.venv/bin/python research/run_ocr_robustness.py
  # Section 5.3 (OCR)
.venv/bin/python research/run_generation_eval.py
  # Section 5.3 (RAGAS)
.venv/bin/python research/run_rvq_analysis.py
  # Section 5.3 (RVQ)
.venv/bin/python research/run_ndsu_reproduction.py
  # Section 5.2.1
.venv/bin/python -m pytest tests/ -q
  # Chapter 4
```

All experiments are seeded (42, 7, 17 as applicable) for reproducibility. Raw logs: `research/experiments/*.json`, indexed in `research/experiments/registry.json`.